WAF 防火墙安全防护验证工具使用手册

2025-03-20

 Copyright © 上海斗象信息科技有限公司 2026 All Rights Reserved

一、工具简介

GoTestWAF 是一款专业的 Web 应用防火墙（WAF）和 API 网关安全测试工具。它通过发送各类恶意请求来评估安全解决方案的防护能力，帮助安全团队发现配置问题并优化安全策略。

主要功能

- 多种攻击类型测试：支持 SQL 注入、XSS、RCE、LFI、XXE 等 OWASP Top 10 攻击
- 多协议支持：HTTP/HTTPS、gRPC、GraphQL
- 多种编码方式：Base64、URL 编码、JSUnicode、XML 实体等
- 多种注入位置：URL 路径、请求参数、请求头、JSON 正文、表单数据等
- OpenAPI 集成：支持基于 OpenAPI 规范自动生成测试请求模板
- 多种报告格式：HTML、PDF、JSON、DOCX（中文版）
- 真实负例测试：验证 WAF 是否存在误报

二、环境要求

系统要求

项目	要求
操作系统	Windows 10/11、Linux、macOS
内存	最低 2GB，推荐 4GB 以上
磁盘空间	100MB 以上
网络	需要访问目标 URL

依赖组件

- PDF 报告生成：需要安装 Chrome/Chromium 浏览器（无头模式）
- gRPC 测试：需要目标服务开放 gRPC 端口
- GraphQL 测试：需要目标服务支持 GraphQL 接口

三、安装部署

3.1 直接使用可执行文件

```
# Windows
gotestwaf.exe --url=https://target-url --noEmailReport

# Linux/macOS
./gotestwaf --url=https://target-url --noEmailReport
```

3.2 从源码编译

```
# 克隆仓库
git clone https://github.com/wallarm/gotestwaf.git
cd gotestwaf

# 编译
go build -o gotestwaf ./cmd/gotestwaf/

# 指定版本号编译
go build -ldflags "-X github.com/wallarm/gotestwaf/internal/version.Version=v0.5.8" -o gotestwaf ./cmd/gotestwaf/
```

四、快速开始

4.1 基础测试

```
# 最简单的测试命令
gotestwaf --url=https://www.example.com --noEmailReport
```

4.2 生成中文 DOCX 报告

Gotestwaf-cn --url=https://www.example.com --reportFormat=docx --noEmailReport

4.3 使用自定义测试用例

```
gotestwaf --url=https://www.example.com \
--testCasesPath=./custom-testcases \
--reportFormat=html,pdf,docx \
--noEmailReport
```

五、命令行参数详解

5.1 基本参数

参数	默认值	说明
【--url】	-	目标 WAF 保护的 URL（必填）
【--wsUrl】	-	WebSocket 目标 URL
【--grpcPort】	0	gRPC 服务端口（0 表示不测试 gRPC）
【--graphqlUrl】	-	GraphQL 端点 URL

5.2 测试配置参数

参数	默认值	说明
【--testCasesPath】	testcases	测试用例目录路径
【--testSet】	-	指定测试集（逗号分隔，如 owasp,owasp-api）
【--testCase】	-	指定测试用例（逗号分隔）
【--openapiFile】	-	OpenAPI 规范文件路径

商密 Copyright © 上海斗象信息科技有限公司 2026 All Rights Reserved

【--blockStatusCodes】	403	WAF 拦截的 HTTP 状态码（逗号分隔）
【--passStatusCodes】	200,301,302,401,404,405	WAF 放行的 HTTP 状态码
【--blockRegex】	-	拦截响应的正则匹配
【--passRegex】	-	放行响应的正则匹配
【--ignoreUnresolved】	false	忽略未确定的测试结果
【--skipWAFIdentification】	false	跳过 WAF 识别

5.3 报告参数

参数	默认值	说明
【--reportFormat】	pdf	报告格式：html, pdf, json, docx
【--reportName】	waf-evaluation-report	报告文件名（不含扩展名）
【--reportPath】	reports	报告输出目录
【--noEmailReport】	false	不发送邮件报告
【--includePayloads】	false	在报告中包含载荷详情

5.4 性能参数

参数	默认值	说明
【--workers】	自动计算	并发工作线程数
【--maxRPS】	0	每秒最大请求数（0 表示无限制）
【--maxRedirects】	50	最大重定向次数
【--idleConnTimeout】	2	空闲连接超时（秒）
【--followCookies】	false	跟随 Set-Cookie

5.5 HTTP 客户端参数

参数	默认值	说明
【--httpTimeout】	15	HTTP 请求超时时间（秒）
【--proxy】	-	HTTP 代理地址
【--tlsVerify】	true	验证 TLS 证书
【--chromeLocation】	自动检测	Chrome 浏览器路径
【--headless】	true	无头模式运行 Chrome

 Copyright © 上海斗象信息科技有限公司 2026 All Rights Reserved

5.6 WAF 配置参数

参数	默认值	说明
【--wafName】	generic	WAF 名称（显示在报告中）
【--openapiFile】	-	OpenAPI 规范文件

六、测试用例说明

6.1 内置测试集

GoTestWAF 提供以下内置测试集：

测试集目录	类型	说明
【testcases/owasp】	真正例	OWASP 常见 Web 攻击
【testcases/owasp-api】	真正例	API 专用攻击测试
【testcases/community】	真正例	社区贡献的攻击载荷
【testcases/false-pos】	真负例	误报测试（正常请求）

6.2 测试用例文件格式

测试用例采用 YAML 格式：

```
payload:
- "<script>alert('XSS')</script>"
- "<img src=x onerror=alert(1)>"
- "javascript:alert(document.cookie)"
encoder:
- Plain
- Base64Flat
- URL
```

 Copyright © 上海斗象信息科技有限公司 2026 All Rights Reserved

```
placeholder:
- Header
- URLParam
- JSONBody
type: XSS
```

6.3 支持的编码器

编码器	说明
Plain	原始载荷，无编码
Base64Flat	Base64 编码（无换行）
URL	URL 编码
JSUnicode	JavaScript Unicode 编码
XMLEntity	XML 实体编码
Base64	标准 Base64 编码

6.4 支持的注入位置

位置	说明
URLPath	URL 路径部分
URLParam	URL 查询参数
Header	HTTP 请求头
JSONBody	JSON 格式请求体
HTMLForm	表单数据
RequestContext	请求上下文
gRPC	gRPC 调用参数
GraphQL	GraphQL 查询参数

6.5 自定义测试用例

在指定目录创建 YAML 文件：

```
mkdir -p custom-testcases/my-attacks
cat > custom-testcases/my-attacks/sql-advanced.yaml << 'EOF'
```

 Copyright © 上海斗象信息科技有限公司 2026 All Rights Reserved

```
payload:
- "' OR '1'='1"
- "1; DROP TABLE users--"
- "UNION SELECT username, password FROM users--"
encoder:
- Plain
- URL
placeholder:
- URLParam
- JSONBody
type: SQL Injection
EOF

# 使用自定义测试用例
gotestwaf --url=https://target.com --testCasesPath=custom-testcases --noEmailReport
```

七、OpenAPI 集成

7.1 功能说明

GoTestWAF 可以解析 OpenAPI 规范文件，自动生成符合 API 定义的测试请求模板，使测试更加真实有效。

7.2 使用方法

```
# 使用 OpenAPI 规范进行测试
gotestwaf --url=https://api.example.com \
--openapiFile=./api-spec.yaml \
--noEmailReport
```

7.3 支持的 OpenAPI 版本

- OpenAPI 3.0.x
- OpenAPI 3.1.x
- Swagger 2.0

八、报告解读

8.1 评分体系

分数范围	等级	风险等级	建议
≥90%	优秀	低风险	WAF 配置优秀，定期复查即可
75-89%	良好	中低风险	建议优化个别规则
60-74%	中等	中风险	需要调优 WAF 规则
40-59%	及格	中高风险	需要重点优化配置
<40%	不及格	高风险	需要重新评估 WAF 方案

8.2 测试指标说明

指标	说明
真正例测试	恶意请求，应被 WAF 拦截
真负例测试	正常请求，应被 WAF 放行
已拦截	WAF 成功拦截的恶意请求
已绕过	WAF 未能拦截的恶意请求
未确定	无法判断是否被拦截
失败	请求发送失败

8.3 报告格式对比

格式	优点	适用场景
HTML	可在浏览器查看，包含图表	快速查看、分享
PDF	打印友好，格式固定	正式报告、存档
DOCX	可编辑，中文版本	报告修改、本地编辑

JSON	结构化数据，易解析	CI/CD 集成、自动化处理

九、高级用法

9.1 多格式报告输出

```
gotestwaf --url=https://target.com \
--reportFormat=html,pdf,docx,json \
--noEmailReport
```

9.2 指定测试范围

```
# 仅测试 XSS 和 SQL 注入
gotestwaf --url=https://target.com \
--testCase=xss,sqli \
--noEmailReport

# 仅测试 OWASP 测试集
gotestwaf --url=https://target.com \
--testSet=owasp \
--noEmailReport
```

9.3 调整性能参数

```
# 高并发测试
gotestwaf --url=https://target.com \
--workers=20 \
--maxRPS=100 \
--noEmailReport

# 低速测试（避免触发限速）
```

```
gotestwaf --url=https://target.com \
--workers=2 \
--maxRPS=10 \
--noEmailReport
```

9.4 使用代理

```
gotestwaf --url=https://target.com \
--proxy=http://127.0.0.1:8080 \
--noEmailReport
```

9.5 自定义 WAF 判断规则

```
# 使用正则表达式判断拦截
gotestwaf --url=https://target.com \
--blockRegex="Access Denied|WAF Blocked" \
--passRegex="Welcome|Success" \
--noEmailReport
```

9.6 gRPC 和 GraphQL 测试

```
# gRPC 测试
gotestwaf --url=https://target.com \
--grpcPort=9000 \
--noEmailReport

# GraphQL 测试
gotestwaf --url=https://target.com \
--graphqlUrl=/graphql \
--noEmailReport
```

十、常见问题

Q1: PDF 报告生成失败

原因：未安装 Chrome 浏览器或路径不正确

解决方案：

```
# 指定 Chrome 路径
gotestwaf --url=https://target.com \
--chromeLocation="C:\Program Files\Google\Chrome\Application\chrome.exe" \
--reportFormat=pdf \
--noEmailReport

# 或使用其他格式
gotestwaf --url=https://target.com \
--reportFormat=html,docx \
--noEmailReport
```

Q2: 测试结果未确定率过高

原因：WAF 判断规则不匹配

解决方案：

```
# 自定义拦截状态码
gotestwaf --url=https://target.com \
--blockStatusCodes=403,406,429,503 \
--noEmailReport

# 使用正则表达式
gotestwaf --url=https://target.com \
--blockRegex="blocked|denied|forbidden" \
--noEmailReport
```

Q3: 连接超时

原因：网络问题或目标响应慢

解决方案：
```
gotestwaf --url=https://target.com \
--httpTimeout=30 \
--idleConnTimeout=5 \
--noEmailReport
```

Q4: 如何测试内网 WAF

解决方案：使用代理或调整网络配置
```
gotestwaf --url=http://192.168.1.100 \
--proxy=http://internal-proxy:8080 \
--tlsVerify=false \
--noEmailReport
```

十一、最佳实践

11.1 测试前准备

确认目标系统允许安全测试
获取必要的授权文件
了解 WAF 产品类型和版本
准备 OpenAPI 规范文件（如有）
配置测试环境网络

11.2 测试执行建议

首次测试使用默认配置，了解基线情况
根据结果调整 【--blockStatusCodes】 和 【--blockRegex】
针对绕过的攻击类型添加自定义测试用例
定期执行测试，跟踪安全状态变化

 Copyright © 上海斗象信息科技有限公司 2026 All Rights Reserved

保存测试报告，建立安全基线

11.3 结果分析建议

重点关注绕过的测试用例
分析误报情况（真负例被拦截）
对照 WAF 日志验证判断准确性
根据结果优化 WAF 规则配置

十二、附录

A. 完整命令示例

```
# 完整测试命令示例
gotestwaf \
--url=https://www.example.com \
--wafName="ModSecurity" \
--testCasesPath=testcases \
--testSet=owasp,community \
--reportFormat=html,pdf,docx,json \
--reportName=example-waf-test \
--reportPath=./reports \
--blockStatusCodes=403,406 \
--workers=10 \
--httpTimeout=20 \
--includePayloads \
--noEmailReport
```

B. 退出码说明

退出码	说明
0	测试成功完成
1	配置错误

2	连接错误
3	测试执行错误
4	报告生成错误

C. 相关资源

- GitHub 仓库：https://github.com/wallarm/gotestwaf
- 问题反馈：https://github.com/wallarm/gotestwaf/issues
- Wallarm 官网：https://wallarm.com

文档版本：v1.0
更新日期：2026 年 3 月 19 日
适用版本：GoTestWAF v0.5.8+

 Copyright © 上海斗象信息科技有限公司 2026 All Rights Reserved